

《部署一套完整的企业级 K8s 集群》

v1.20, 二进制方式

作者信息	李振良（阿良），微信：k8init
DevOps 实战学院	http://www.aliangedu.cn
说明	该文档有导航窗格，方便阅读，如果左侧没有显示，请检查 word 是否启用。 转载请注明作者，拒绝不道德行为！
一键部署脚本	https://github.com/lizhenliang/ansible-install-k8s
最后更新时间	2021-04-06



阿良个人微信



DevOps技术栈公众号

一、前置知识点

1.1 生产环境部署 K8s 集群的两种方式

- **kubeadm**

Kubeadm 是一个 K8s 部署工具，提供 `kubeadm init` 和 `kubeadm join`，用于快速部署 Kubernetes 集群。

- **二进制包**

从 github 下载发行版的二进制包，手动部署每个组件，组成 Kubernetes 集群。

小结：Kubeadm 降低部署门槛，但屏蔽了很多细节，遇到问题很难排查。如果想更容易可控，推荐使用二进制包部署 Kubernetes 集群，虽然手动部署麻烦点，期间可以学习很多工作原理，也利于后期维护。

1.2 准备环境

服务器要求：

- 建议最小硬件配置：2 核 CPU、2G 内存、30G 硬盘
- 服务器最好可以访问外网，会有从网上拉取镜像需求，如果服务器不能上网，需要提前下载对应镜像并导入节点

软件环境：

软件	版本
操作系统	CentOS7.x_x64 (mini)
容器引擎	Docker CE 19
Kubernetes	Kubernetes v1.20

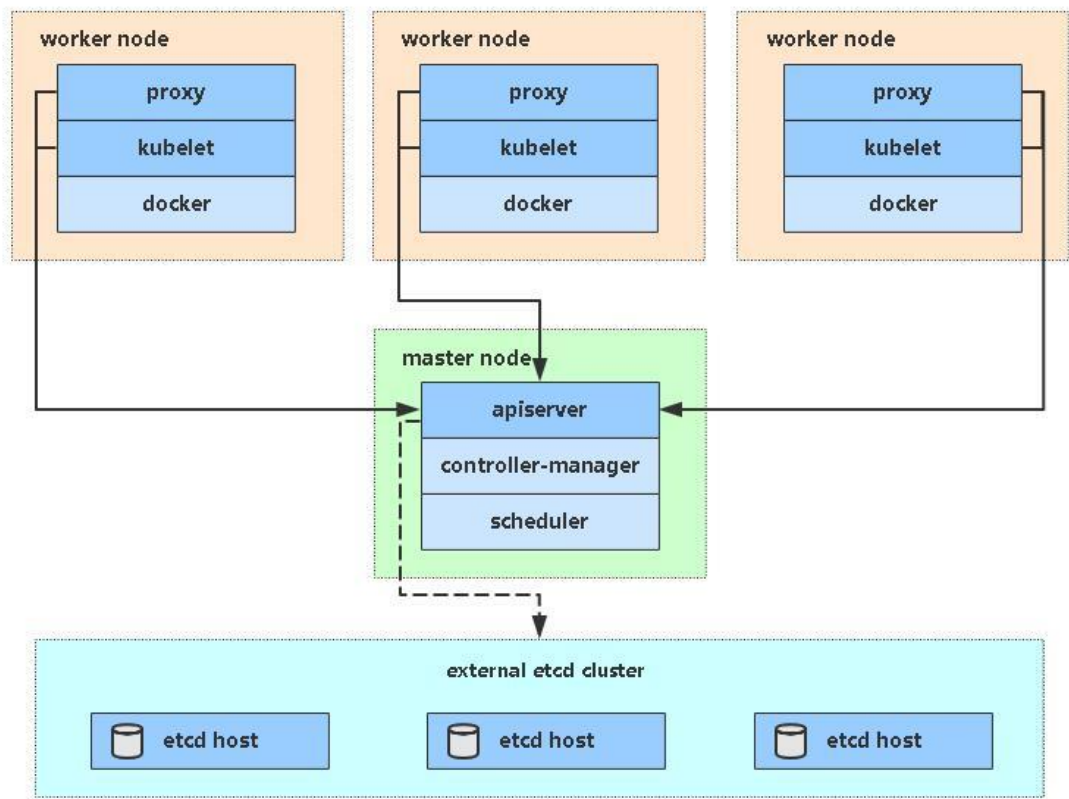
服务器整体规划：

角色	IP	组件
k8s-master1	192.168.31.71	kube-apiserver , kube-controller-manager , kube-scheduler , kubelet , kube-proxy , docker, etcd,

		nginx, keepalived
k8s-master2	192.168.31.74	kube-apiserver , kube-controller-manager , kube-scheduler , kubelet , kube-proxy , docker, nginx, keepalived
k8s-node1	192.168.31.72	kubelet, kube-proxy, docker, etcd
k8s-node2	192.168.31.73	kubelet, kube-proxy, docker, etcd
负载均衡器 IP	192.168.31.88 (VIP)	

须知：考虑到有些朋友电脑配置较低，一次性开四台机器会跑不动，所以搭建这套 K8s 高可用集群分两部分实施，先部署一套单 Master 架构（3 台），再扩容为多 Master 架构（4 台或 6 台），顺便再熟悉下 Master 扩容流程。

单 Master 架构图：



单 Master 服务器规划:

角色	IP	组件
k8s-master	192.168.31.71	kube-apiserver , kube-controller-manager , kube-scheduler, etcd
k8s-node1	192.168.31.72	kubelet, kube-proxy, docker, etcd
k8s-node2	192.168.31.73	kubelet, kube-proxy, docker, etcd

1.3 操作系统初始化配置

```
# 关闭防火墙
systemctl stop firewalld
systemctl disable firewalld

# 关闭 selinux
```

```
sed -i 's/enforcing/disabled/' /etc/selinux/config # 永久
setenforce 0 # 临时

# 关闭 swap
swapoff -a # 临时
sed -ri 's/.*swap.*/#&/' /etc/fstab # 永久

# 根据规划设置主机名
hostnamectl set-hostname <hostname>

# 在 master 添加 hosts
cat >> /etc/hosts << EOF
192.168.31.71 k8s-master1
192.168.31.72 k8s-node1
192.168.31.73 k8s-node2
EOF

# 将桥接的 IPv4 流量传递到 iptables 的链
cat > /etc/sysctl.d/k8s.conf << EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF
sysctl --system # 生效

# 时间同步
yum install ntpdate -y
ntpdate time.windows.com
```

二、部署 Etcd 集群

Etcd 是一个分布式键值存储系统, Kubernetes 使用 Etcd 进行数据存储, 所以先准备一个 Etcd 数据库, 为解决 Etcd 单点故障, 应采用集群方式部署, 这里使用 3 台组建集群, 可容忍 1 台机器故障, 当然, 你也可以使用 5 台组建集群, 可容忍 2 台机器故障。

节点名称	IP
etcd-1	192.168.31.71
etcd-2	192.168.31.72

etcd-3	192.168.31.73
--------	---------------

注: 为了节省机器, 这里与 K8s 节点机器复用。也可以独立于 k8s 集群之外部署, 只要 apiserver 能连接到就行。

2.1 准备 cfssl 证书生成工具

cfssl 是一个开源的证书管理工具, 使用 json 文件生成证书, 相比 openssl 更方便使用。

找任意一台服务器操作, 这里用 Master 节点。

```
wget https://pkg.cfssl.org/R1.2/cfssl_linux-amd64
wget https://pkg.cfssl.org/R1.2/cfssljson_linux-amd64
wget https://pkg.cfssl.org/R1.2/cfssl-certinfo_linux-amd64
chmod +x cfssl_linux-amd64 cfssljson_linux-amd64 cfssl-certinfo_linux-amd64
mv cfssl_linux-amd64 /usr/local/bin/cfssl
mv cfssljson_linux-amd64 /usr/local/bin/cfssljson
mv cfssl-certinfo_linux-amd64 /usr/bin/cfssl-certinfo
```

2.2 生成 Etcd 证书

1. 自签证书颁发机构 (CA)

创建工作目录:

```
mkdir -p ~/TLS/{etcd,k8s}

cd ~/TLS/etcd
```

自签 CA:

```
cat > ca-config.json << EOF
{
  "signing": {
    "default": {
      "expiry": "87600h"
    },
    "profiles": {
      "www": {
        "expiry": "87600h",
```

```
        "usages": [
            "signing",
            "key encipherment",
            "server auth",
            "client auth"
        ]
    }
}
}
EOF
```

```
cat > ca-csr.json << EOF
{
    "CN": "etcd CA",
    "key": {
        "algo": "rsa",
        "size": 2048
    },
    "names": [
        {
            "C": "CN",
            "L": "Beijing",
            "ST": "Beijing"
        }
    ]
}
EOF
```

生成证书:

```
cfssl gencert -initca ca-csr.json | cfssljson -bare ca -
```

会生成 ca.pem 和 ca-key.pem 文件。

2. 使用自签 CA 签发 Etcd HTTPS 证书

创建证书申请文件:

```
cat > server-csr.json << EOF
{
  "CN": "etcd",
  "hosts": [
    "192.168.31.71",
    "192.168.31.72",
    "192.168.31.73"
  ],
  "key": {
    "algo": "rsa",
    "size": 2048
  },
  "names": [
    {
      "C": "CN",
      "L": "BeiJing",
      "ST": "BeiJing"
    }
  ]
}
EOF
```

注：上述文件 hosts 字段中 IP 为所有 etcd 节点的集群内部通信 IP，一个都不能少！为了方便后期扩容可以多写几个预留的 IP。

生成证书：

```
cfssl gencert -ca=ca.pem -ca-key=ca-key.pem -config=ca-config.json -profile=www
server-csr.json | cfssljson -bare server
```

会生成 server.pem 和 server-key.pem 文件。

2.3 从 Github 下载二进制文件

下载地址：<https://github.com/etcd-io/etcd/releases/download/v3.4.9/etcd-v3.4.9-linux-amd64.tar.gz>

2.4 部署 Etcd 集群

以下在节点 1 上操作，为简化操作，待会将节点 1 生成的所有文件拷贝到节点 2 和节点 3。

1. 创建工作目录并解压二进制包

```
mkdir /opt/etcd/{bin,cfg,ssl} -p
tar zxvf etcd-v3.4.9-linux-amd64.tar.gz
mv etcd-v3.4.9-linux-amd64/{etcd,etcdctl} /opt/etcd/bin/
```

2. 创建 etcd 配置文件

```
cat > /opt/etcd/cfg/etcd.conf << EOF
#[Member]
ETCD_NAME="etcd-1"
ETCD_DATA_DIR="/var/lib/etcd/default.etcd"
ETCD_LISTEN_PEER_URLS="https://192.168.31.71:2380"
ETCD_LISTEN_CLIENT_URLS="https://192.168.31.71:2379"

#[Clustering]
ETCD_INITIAL_ADVERTISE_PEER_URLS="https://192.168.31.71:2380"
ETCD_ADVERTISE_CLIENT_URLS="https://192.168.31.71:2379"
ETCD_INITIAL_CLUSTER="etcd-1=https://192.168.31.71:2380,etcd-2=https://192.168.31.72:2380,etcd-3=https://192.168.31.73:2380"
ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster"
ETCD_INITIAL_CLUSTER_STATE="new"
EOF
```

- ETCD_NAME: 节点名称, 集群中唯一
- ETCD_DATA_DIR: 数据目录
- ETCD_LISTEN_PEER_URLS: 集群通信监听地址
- ETCD_LISTEN_CLIENT_URLS: 客户端访问监听地址
- ETCD_INITIAL_ADVERTISE_PEERURLS: 集群通告地址
- ETCD_ADVERTISE_CLIENT_URLS: 客户端通告地址
- ETCD_INITIAL_CLUSTER: 集群节点地址
- ETCD_INITIALCLUSTER_TOKEN: 集群 Token
- ETCD_INITIALCLUSTER_STATE: 加入集群的当前状态, new 是新集群, existing 表示加入已有集群

3. systemd 管理 etcd

```
cat > /usr/lib/systemd/system/etcd.service << EOF
[Unit]
Description=Etcd Server
After=network.target
After=network-online.target
Wants=network-online.target

[Service]
Type=notify
EnvironmentFile=/opt/etcd/cfg/etcd.conf
ExecStart=/opt/etcd/bin/etcd \
--cert-file=/opt/etcd/ssl/server.pem \
--key-file=/opt/etcd/ssl/server-key.pem \
--peer-cert-file=/opt/etcd/ssl/server.pem \
--peer-key-file=/opt/etcd/ssl/server-key.pem \
--trusted-ca-file=/opt/etcd/ssl/ca.pem \
--peer-trusted-ca-file=/opt/etcd/ssl/ca.pem \
--logger=zap
Restart=on-failure
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
EOF
```

4. 拷贝刚才生成的证书

把刚才生成的证书拷贝到配置文件中的路径:

```
cp ~/TLS/etcd/ca.pem ~/TLS/etcd/server.pem /opt/etcd/ssl/
```

5. 启动并设置开机启动

```
systemctl daemon-reload
systemctl start etcd
systemctl enable etcd
```

6. 将上面节点 1 所有生成的文件拷贝到节点 2 和节点 3

```
scp -r /opt/etcd/ root@192.168.31.72:/opt/  
scp /usr/lib/systemd/system/etcd.service root@192.168.31.72:/usr/lib/systemd/system/  
scp -r /opt/etcd/ root@192.168.31.73:/opt/  
scp /usr/lib/systemd/system/etcd.service root@192.168.31.73:/usr/lib/systemd/system/
```

然后在节点 2 和节点 3 分别修改 etcd.conf 配置文件中的节点名称和当前服务器 IP:

```
vi /opt/etcd/cfg/etcd.conf  
#[Member]  
ETCD_NAME="etcd-1" # 修改此处, 节点 2 改为 etcd-2, 节点 3 改为 etcd-3  
ETCD_DATA_DIR="/var/lib/etcd/default.etcd"  
ETCD_LISTEN_PEER_URLS="https://192.168.31.71:2380" # 修改此处为当前服务器 IP  
ETCD_LISTEN_CLIENT_URLS="https://192.168.31.71:2379" # 修改此处为当前服务器 IP  
  
#[Clustering]  
ETCD_INITIAL_ADVERTISE_PEER_URLS="https://192.168.31.71:2380" # 修改此处为当前服务器 IP  
ETCD_ADVERTISE_CLIENT_URLS="https://192.168.31.71:2379" # 修改此处为当前服务器 IP  
ETCD_INITIAL_CLUSTER="etcd-1=https://192.168.31.71:2380,etcd-2=https://192.168.31.72:2380,etcd-3=https://192.168.31.73:2380"  
ETCD_INITIAL_CLUSTER_TOKEN="etcd-cluster"  
ETCD_INITIAL_CLUSTER_STATE="new"
```

最后启动 etcd 并设置开机启动, 同上。

7. 查看集群状态

```
ETCDCTL_API=3 /opt/etcd/bin/etcdctl --cacert=/opt/etcd/ssl/ca.pem --  
cert=/opt/etcd/ssl/server.pem --key=/opt/etcd/ssl/server-key.pem --  
endpoints="https://192.168.31.71:2379,https://192.168.31.72:2379,https://192.168.31.73:2379" endpoint health --write-out=table
```

ENDPOINT	HEALTH	TOOK	ERROR
https://192.168.31.71:2379	true	10.301506ms	
https://192.168.31.73:2379	true	12.87467ms	

```
| https://192.168.31.72:2379 | true | 13.225954ms | |  
+-----+-----+-----+-----+
```

如果输出上面信息，就说明集群部署成功。

如果有问题第一步先看日志：/var/log/message 或 journalctl -u etcd

三、安装 Docker

这里使用 Docker 作为容器引擎，也可以换成别的，例如 containerd

下载地址：https://download.docker.com/linux/static/stable/x86_64/docker-19.03.9.tgz

以下在所有节点操作。这里采用二进制安装，用 yum 安装也一样。

3.1 解压二进制包

```
tar zxvf docker-19.03.9.tgz  
mv docker/* /usr/bin
```

3.2 systemd 管理 docker

```
cat > /usr/lib/systemd/system/docker.service << EOF  
[Unit]  
Description=Docker Application Container Engine  
Documentation=https://docs.docker.com  
After=network-online.target firewalld.service  
Wants=network-online.target  
  
[Service]  
Type=notify  
ExecStart=/usr/bin/dockerd  
ExecReload=/bin/kill -s HUP $MAINPID  
LimitNOFILE=infinity  
LimitNPROC=infinity  
LimitCORE=infinity  
TimeoutStartSec=0  
Delegate=yes  
KillMode=process  
Restart=on-failure  
StartLimitBurst=3
```

```
StartLimitInterval=60s
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

3.3 创建配置文件

```
mkdir /etc/docker
```

```
cat > /etc/docker/daemon.json << EOF
```

```
{  
  "registry-mirrors": ["https://b9pmyelo.mirror.aliyuncs.com"]  
}  
EOF
```

- registry-mirrors 阿里云镜像加速器

3.4 启动并设置开机启动

```
systemctl daemon-reload
```

```
systemctl start docker
```

```
systemctl enable docker
```

四、部署 Master Node

如果你在学习中遇到问题或者文档有误可联系阿良~ 微信: k8init

4.1 生成 kube-apiserver 证书

1. 自签证书颁发机构 (CA)

```
cd ~/TLS/k8s
```

```
cat > ca-config.json << EOF
```

```
{  
  "signing": {  
    "default": {  
      "expiry": "87600h"  
    },  
    "profiles": {  
      "kubernetes": {  
        "expiry": "87600h",  
        "usages": [  

```

```
        "signing",
        "key encipherment",
        "server auth",
        "client auth"
    ]
}
}
}
EOF
cat > ca-csr.json << EOF
{
    "CN": "kubernetes",
    "key": {
        "algo": "rsa",
        "size": 2048
    },
    "names": [
        {
            "C": "CN",
            "L": "Beijing",
            "ST": "Beijing",
            "O": "k8s",
            "OU": "System"
        }
    ]
}
EOF
```

生成证书:

```
cfssl gencert -initca ca-csr.json | cfssljson -bare ca -
```

会生成 ca.pem 和 ca-key.pem 文件。

2. 使用自签 CA 签发 kube-apiserver HTTPS 证书

创建证书申请文件:

```
cat > server-csr.json << EOF
{
```

```
"CN": "kubernetes",
"hosts": [
    "10.0.0.1",
    "127.0.0.1",
    "192.168.31.71",
    "192.168.31.72",
    "192.168.31.73",
    "192.168.31.74",
    "192.168.31.88",
    "kubernetes",
    "kubernetes.default",
    "kubernetes.default.svc",
    "kubernetes.default.svc.cluster",
    "kubernetes.default.svc.cluster.local"
],
"key": {
    "algo": "rsa",
    "size": 2048
},
"names": [
    {
        "C": "CN",
        "L": "BeiJing",
        "ST": "BeiJing",
        "O": "k8s",
        "OU": "System"
    }
]
}
EOF
```

注：上述文件 hosts 字段中 IP 为所有 Master/LB/VIP IP，一个都不能少！为了方便后期扩容可以多写几个预留的 IP。

生成证书：

```
cfssl gencert -ca=ca.pem -ca-key=ca-key.pem -config=ca-config.json -
profile=kubernetes server-csr.json | cfssljson -bare server
```

会生成 server.pem 和 server-key.pem 文件。

4.2 从 Github 下载二进制文件

下载地址:

<https://github.com/kubernetes/kubernetes/blob/master/CHANGELOG/CHANGELOG-1.20.md>

注: 打开链接你会发现里面有很多包, 下载一个 server 包就够了, 包含了 Master 和 Worker Node 二进制文件。

4.3 解压二进制包

```
mkdir -p /opt/kubernetes/{bin,cfg,ssl,logs}
tar zxvf kubernetes-server-linux-amd64.tar.gz
cd kubernetes/server/bin
cp kube-apiserver kube-scheduler kube-controller-manager /opt/kubernetes/bin
cp kubectl /usr/bin/
```

4.4 部署 kube-apiserver

1. 创建配置文件

```
cat > /opt/kubernetes/cfg/kube-apiserver.conf << EOF
KUBE_APISERVER_OPTS="--logtostderr=false \\\n--v=2 \\\n--log-dir=/opt/kubernetes/logs \\\n--etcd-\\nservers=https://192.168.31.71:2379,https://192.168.31.72:2379,https://192.168.31.73:2379 \\\n--bind-address=192.168.31.71 \\\n--secure-port=6443 \\\n--advertise-address=192.168.31.71 \\\n--allow-privileged=true \\\n--service-cluster-ip-range=10.0.0.0/24 \\\n--enable-admission-\\nplugins=NamespaceLifecycle,LimitRanger,ServiceAccount,ResourceQuota,NodeRestriction \\\n--authorization-mode=RBAC,Node \\\n--enable-bootstrap-token-auth=true \\\n--token-auth-file=/opt/kubernetes/cfg/token.csv \\\n--service-node-port-range=30000-32767 \\\n--kubelet-client-certificate=/opt/kubernetes/ssl/server.pem \\\n--kubelet-client-key=/opt/kubernetes/ssl/server-key.pem \\\n"
```



```
--tls-cert-file=/opt/kubernetes/ssl/server.pem \\  
--tls-private-key-file=/opt/kubernetes/ssl/server-key.pem \\  
--client-ca-file=/opt/kubernetes/ssl/ca.pem \\  
--service-account-key-file=/opt/kubernetes/ssl/ca-key.pem \\  
--service-account-issuer=api \\  
--service-account-signing-key-file=/opt/kubernetes/ssl/server-key.pem \\  
--etcd-cafile=/opt/etcd/ssl/ca.pem \\  
--etcd-certfile=/opt/etcd/ssl/server.pem \\  
--etcd-keyfile=/opt/etcd/ssl/server-key.pem \\  
--requestheader-client-ca-file=/opt/kubernetes/ssl/ca.pem \\  
--proxy-client-cert-file=/opt/kubernetes/ssl/server.pem \\  
--proxy-client-key-file=/opt/kubernetes/ssl/server-key.pem \\  
--requestheader-allowed-names=kubernetes \\  
--requestheader-extra-headers-prefix=X-Remote-Extra- \\  
--requestheader-group-headers=X-Remote-Group \\  
--requestheader-username-headers=X-Remote-User \\  
--enable-aggregator-routing=true \\  
--audit-log-maxage=30 \\  
--audit-log-maxbackup=3 \\  
--audit-log-maxsize=100 \\  
--audit-log-path=/opt/kubernetes/logs/k8s-audit.log"  
EOF
```

注：上面两个\ \ 第一个是转义符，第二个是换行符，使用转义符是为了使用 EOF 保留换行符。

- --logtostderr: 启用日志
- ---v: 日志等级
- --log-dir: 日志目录
- --etcd-servers: etcd 集群地址
- --bind-address: 监听地址
- --secure-port: https 安全端口
- --advertise-address: 集群通告地址
- --allow-privileged: 启用授权
- --service-cluster-ip-range: Service 虚拟 IP 地址段

- `--enable-admission-plugins`: 准入控制模块
- `--authorization-mode`: 认证授权, 启用 RBAC 授权和节点自我管理
- `--enable-bootstrap-token-auth`: 启用 TLS bootstrap 机制
- `--token-auth-file`: bootstrap token 文件
- `--service-node-port-range`: Service nodeport 类型默认分配端口范围
- `--kubelet-client-xxx`: apiserver 访问 kubelet 客户端证书
- `--tls-xxx-file`: apiserver https 证书
- 1.20 版本必须加的参数: `--service-account-issuer`, `--service-account-signing-key-file`
- `--etcd-xxxfile`: 连接 Etcd 集群证书
- `--audit-log-xxx`: 审计日志
- 启动聚合层相关配置: `--requestheader-client-ca-file`, `--proxy-client-cert-file`, `--proxy-client-key-file`, `--requestheader-allowed-names`, `--requestheader-extra-headers-prefix`, `--requestheader-group-headers`, `--requestheader-username-headers`, `--enable-aggregator-routing`

2. 拷贝刚才生成的证书

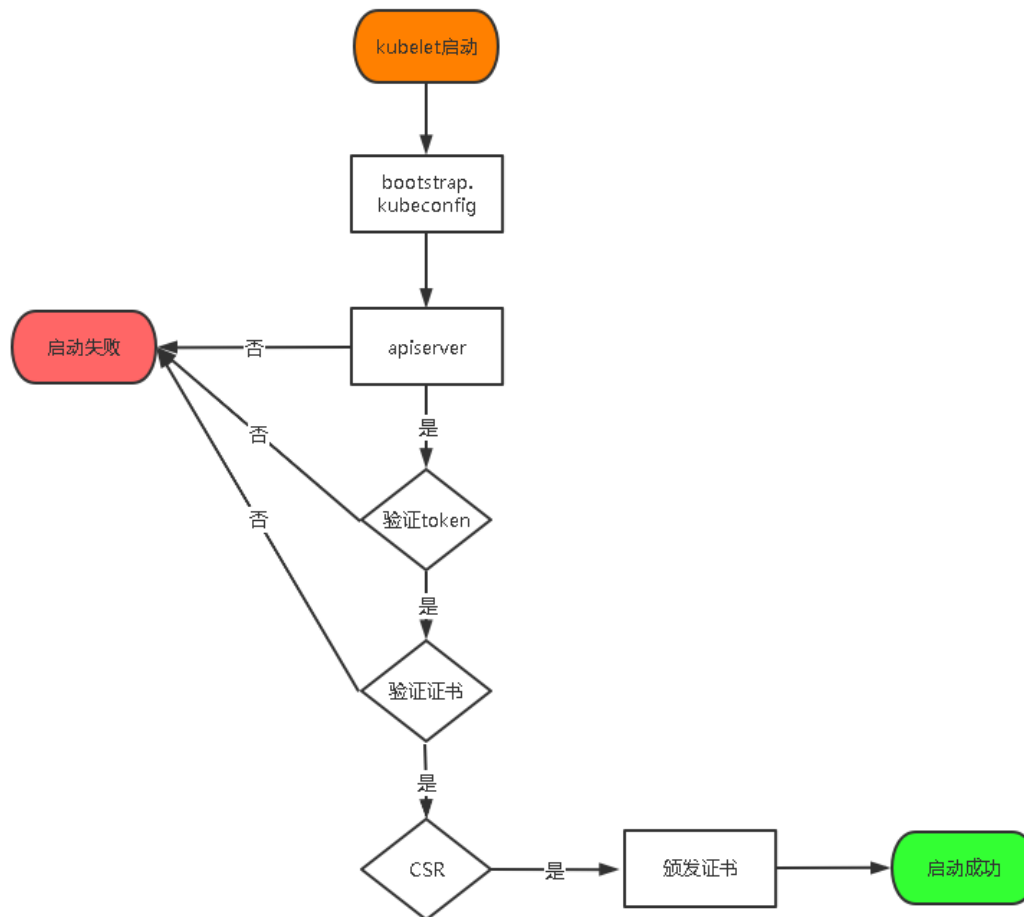
把刚才生成的证书拷贝到配置文件中的路径:

```
cp ~/TLS/k8s/ca.pem ~/TLS/k8s/server.pem /opt/kubernetes/ssl/
```

3. 启用 TLS Bootstrapping 机制

TLS Bootstrapping: Master apiserver 启用 TLS 认证后, Node 节点 kubelet 和 kube-proxy 要与 kube-apiserver 进行通信, 必须使用 CA 签发的有效证书才可以, 当 Node 节点很多时, 这种客户端证书颁发需要大量工作, 同样也会增加集群扩展复杂度。为了简化流程, Kubernetes 引入了 TLS bootstrapping 机制来自动颁发客户端证书, kubelet 会以一个低权限用户自动向 apiserver 申请证书, kubelet 的证书由 apiserver 动态签署。所以强烈建议在 Node 上使用这种方式, 目前主要用于 kubelet, kube-proxy 还是由我们统一颁发一个证书。

TLS bootstrapping 工作流程:



创建上述配置文件中 token 文件:

```
cat > /opt/kubernetes/cfg/token.csv << EOF
c47ffb939f5ca36231d9e3121a252940,kubelet-bootstrap,10001,"system:node-bootstrapper"
EOF
```

格式: token, 用户名, UID, 用户组

token 也可自行生成替换:

```
head -c 16 /dev/urandom | od -An -t x | tr -d ' '
```

4. systemd 管理 apiserver

```
cat > /usr/lib/systemd/system/kube-apiserver.service << EOF
[Unit]
Description=Kubernetes API Server
Documentation=https://github.com/kubernetes/kubernetes

[Service]
EnvironmentFile=/opt/kubernetes/cfg/kube-apiserver.conf
ExecStart=/opt/kubernetes/bin/kube-apiserver $KUBE_APISERVER_OPTS
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF
```

5. 启动并设置开机启动

```
systemctl daemon-reload
systemctl start kube-apiserver
systemctl enable kube-apiserver
```

4.5 部署 kube-controller-manager

1. 创建配置文件

```
cat > /opt/kubernetes/cfg/kube-controller-manager.conf << EOF
KUBE_CONTROLLER_MANAGER_OPTS="--logtostderr=false \\\n--v=2 \\\n--log-dir=/opt/kubernetes/logs \\\n--leader-elect=true \\\n--kubeconfig=/opt/kubernetes/cfg/kube-controller-manager.kubeconfig \\\n--bind-address=127.0.0.1 \\\n--allocate-node-cidrs=true \\\n--cluster-cidr=10.244.0.0/16 \\\n--service-cluster-ip-range=10.0.0.0/24 \\\n--cluster-signing-cert-file=/opt/kubernetes/ssl/ca.pem \\\n--cluster-signing-key-file=/opt/kubernetes/ssl/ca-key.pem \\\n--root-ca-file=/opt/kubernetes/ssl/ca.pem \\\n--service-account-private-key-file=/opt/kubernetes/ssl/ca-key.pem \\\n--cluster-signing-duration=87600h0m0s"\nEOF
```

- `--kubeconfig`: 连接 apiserver 配置文件
- `--leader-elect`: 当该组件启动多个时, 自动选举 (HA)
- `--cluster-signing-cert-file/--cluster-signing-key-file`: 自动为 kubelet 颁发证书的 CA, 与 apiserver 保持一致

2. 生成 kubeconfig 文件

生成 kube-controller-manager 证书:

```
# 切换工作目录
cd ~/TLS/k8s

# 创建证书请求文件
cat > kube-controller-manager-csr.json << EOF
{
  "CN": "system:kube-controller-manager",
  "hosts": [],
  "key": {
    "algo": "rsa",
    "size": 2048
  },
  "names": [
    {
      "C": "CN",
      "L": "BeiJing",
      "ST": "BeiJing",
      "O": "system:masters",
      "OU": "System"
    }
  ]
}
EOF

# 生成证书
cfssl gencert -ca=ca.pem -ca-key=ca-key.pem -config=ca-config.json -
profile=kubernetes kube-controller-manager-csr.json | cfssljson -bare kube-
controller-manager
```

生成 kubeconfig 文件 (以下是 shell 命令, 直接在终端执行):

```
KUBE_CONFIG="/opt/kubernetes/cfg/kube-controller-manager.kubeconfig"
KUBE_APISERVER="https://192.168.31.71:6443"
```

```
kubect1 config set-cluster kubernetes \
  --certificate-authority=/opt/kubernetes/ssl/ca.pem \
  --embed-certs=true \
  --server=${KUBE_APISERVER} \
  --kubeconfig=${KUBE_CONFIG}
kubect1 config set-credentials kube-controller-manager \
  --client-certificate=./kube-controller-manager.pem \
  --client-key=./kube-controller-manager-key.pem \
  --embed-certs=true \
  --kubeconfig=${KUBE_CONFIG}
kubect1 config set-context default \
  --cluster=kubernetes \
  --user=kube-controller-manager \
  --kubeconfig=${KUBE_CONFIG}
kubect1 config use-context default --kubeconfig=${KUBE_CONFIG}
```

3. systemd 管理 controller-manager

```
cat > /usr/lib/systemd/system/kube-controller-manager.service << EOF
[Unit]
Description=Kubernetes Controller Manager
Documentation=https://github.com/kubernetes/kubernetes

[Service]
EnvironmentFile=/opt/kubernetes/cfg/kube-controller-manager.conf
ExecStart=/opt/kubernetes/bin/kube-controller-manager ${KUBE_CONTROLLER_MANAGER_OPTS}
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF
```

4. 启动并设置开机启动

```
systemctl daemon-reload
systemctl start kube-controller-manager
systemctl enable kube-controller-manager
```

4.6 部署 kube-scheduler

1. 创建配置文件

```
cat > /opt/kubernetes/cfg/kube-scheduler.conf << EOF
KUBE_SCHEDULER_OPTS="--logtostderr=false \"
--v=2 \"
--log-dir=/opt/kubernetes/logs \"
--leader-elect \"
--kubeconfig=/opt/kubernetes/cfg/kube-scheduler.kubeconfig \"
--bind-address=127.0.0.1\"
EOF
```

- --kubeconfig: 连接 apiserver 配置文件
- --leader-elect: 当该组件启动多个时, 自动选举 (HA)

2. 生成 kubeconfig 文件

生成 kube-scheduler 证书:

```
# 切换工作目录
cd ~/TLS/k8s

# 创建证书请求文件
cat > kube-scheduler-csr.json << EOF
{
  "CN": "system:kube-scheduler",
  "hosts": [],
  "key": {
    "algo": "rsa",
    "size": 2048
  },
  "names": [
    {
      "C": "CN",
      "L": "BeiJing",
      "ST": "BeiJing",
      "O": "system:masters",
      "OU": "System"
    }
  ]
}
```

```
}
EOF

# 生成证书
cfssl gencert -ca=ca.pem -ca-key=ca-key.pem -config=ca-config.json -
profile=kubernetes kube-scheduler-csr.json | cfssljson -bare kube-scheduler
```

生成 kubeconfig 文件:

```
KUBE_CONFIG="/opt/kubernetes/cfg/kube-scheduler.kubeconfig"
KUBE_APISERVER="https://192.168.31.71:6443"

kubectl config set-cluster kubernetes \
  --certificate-authority=/opt/kubernetes/ssl/ca.pem \
  --embed-certs=true \
  --server=${KUBE_APISERVER} \
  --kubeconfig=${KUBE_CONFIG}
kubectl config set-credentials kube-scheduler \
  --client-certificate=./kube-scheduler.pem \
  --client-key=./kube-scheduler-key.pem \
  --embed-certs=true \
  --kubeconfig=${KUBE_CONFIG}
kubectl config set-context default \
  --cluster=kubernetes \
  --user=kube-scheduler \
  --kubeconfig=${KUBE_CONFIG}
kubectl config use-context default --kubeconfig=${KUBE_CONFIG}
```

3. systemd 管理 scheduler

```
cat > /usr/lib/systemd/system/kube-scheduler.service << EOF
[Unit]
Description=Kubernetes Scheduler
Documentation=https://github.com/kubernetes/kubernetes

[Service]
EnvironmentFile=/opt/kubernetes/cfg/kube-scheduler.conf
ExecStart=/opt/kubernetes/bin/kube-scheduler ${KUBE_SCHEDULER_OPTS}
Restart=on-failure

[Install]
```



```
WantedBy=multi-user.target
EOF
```

4. 启动并设置开机启动

```
systemctl daemon-reload
systemctl start kube-scheduler
systemctl enable kube-scheduler
```

5. 查看集群状态

生成 kubectl 连接集群的证书:

```
cat > admin-csr.json <<EOF
{
  "CN": "admin",
  "hosts": [],
  "key": {
    "algo": "rsa",
    "size": 2048
  },
  "names": [
    {
      "C": "CN",
      "L": "BeiJing",
      "ST": "BeiJing",
      "O": "system:masters",
      "OU": "System"
    }
  ]
}
EOF
```

```
cfssl gencert -ca=ca.pem -ca-key=ca-key.pem -config=ca-config.json -
profile=kubernetes admin-csr.json | cfssljson -bare admin
```

生成 kubeconfig 文件:

```
mkdir /root/.kube
```

```
KUBE_CONFIG="/root/.kube/config"
KUBE_APISERVER="https://192.168.31.71:6443"

kubectl config set-cluster kubernetes \
  --certificate-authority=/opt/kubernetes/ssl/ca.pem \
  --embed-certs=true \
  --server=${KUBE_APISERVER} \
  --kubeconfig=${KUBE_CONFIG}
kubectl config set-credentials cluster-admin \
  --client-certificate=./admin.pem \
  --client-key=./admin-key.pem \
  --embed-certs=true \
  --kubeconfig=${KUBE_CONFIG}
kubectl config set-context default \
  --cluster=kubernetes \
  --user=cluster-admin \
  --kubeconfig=${KUBE_CONFIG}
kubectl config use-context default --kubeconfig=${KUBE_CONFIG}
```

通过 kubectl 工具查看当前集群组件状态:

```
kubectl get cs
```

NAME	STATUS	MESSAGE	ERROR
scheduler	Healthy	ok	
controller-manager	Healthy	ok	
etcd-2	Healthy	{"health": "true"}	
etcd-1	Healthy	{"health": "true"}	
etcd-0	Healthy	{"health": "true"}	

如上输出说明 Master 节点组件运行正常。

6. 授权 kubelet-bootstrap 用户允许请求证书

```
kubectl create clusterrolebinding kubelet-bootstrap \
  --clusterrole=system:node-bootstrapper \
  --user=kubelet-bootstrap
```

五、部署 Worker Node

如果你在学习遇到问题或者文档有误可联系阿良~ 微信: k8init

下面还是在 Master Node 上操作, 即同时作为 Worker Node

5.1 创建工作目录并拷贝二进制文件

在所有 worker node 创建工作目录:

```
mkdir -p /opt/kubernetes/{bin,cfg,ssl,logs}
```

从 master 节点拷贝:

```
cd kubernetes/server/bin  
cp kubelet kube-proxy /opt/kubernetes/bin # 本地拷贝
```

5.2 部署 kubelet

1. 创建配置文件

```
cat > /opt/kubernetes/cfg/kubelet.conf << EOF  
KUBELET_OPTS="--logtostderr=false \<\  
--v=2 \<\  
--log-dir=/opt/kubernetes/logs \<\  
--hostname-override=k8s-master1 \<\  
--network-plugin=cni \<\  
--kubeconfig=/opt/kubernetes/cfg/kubelet.kubeconfig \<\  
--bootstrap-kubeconfig=/opt/kubernetes/cfg/bootstrap.kubeconfig \<\  
--config=/opt/kubernetes/cfg/kubelet-config.yml \<\  
--cert-dir=/opt/kubernetes/ssl \<\  
--pod-infra-container-image=lizhenliang/pause-amd64:3.0"  
EOF
```

- `--hostname-override`: 显示名称, 集群中唯一
- `--network-plugin`: 启用 CNI
- `--kubeconfig`: 空路径, 会自动生成, 后面用于连接 apiserver
- `--bootstrap-kubeconfig`: 首次启动向 apiserver 申请证书
- `--config`: 配置参数文件
- `--cert-dir`: kubelet 证书生成目录
- `--pod-infra-container-image`: 管理 Pod 网络容器的镜像

2. 配置参数文件

```
cat > /opt/kubernetes/cfg/kubelet-config.yml << EOF
kind: KubeletConfiguration
apiVersion: kubelet.config.k8s.io/v1beta1
address: 0.0.0.0
port: 10250
readOnlyPort: 10255
cgroupDriver: cgroupfs
clusterDNS:
- 10.0.0.2
clusterDomain: cluster.local
failSwapOn: false
authentication:
  anonymous:
    enabled: false
  webhook:
    cacheTTL: 2m0s
    enabled: true
  x509:
    clientCAFile: /opt/kubernetes/ssl/ca.pem
authorization:
  mode: Webhook
  webhook:
    cacheAuthorizedTTL: 5m0s
    cacheUnauthorizedTTL: 30s
evictionHard:
  imagefs.available: 15%
  memory.available: 100Mi
  nodefs.available: 10%
  nodefs.inodesFree: 5%
maxOpenFiles: 1000000
maxPods: 110
EOF
```

3. 生成 kubelet 初次加入集群引导 kubeconfig 文件

```
KUBE_CONFIG="/opt/kubernetes/cfg/bootstrap.kubeconfig"
KUBE_APISERVER="https://192.168.31.71:6443" # apiserver IP:PORT
TOKEN="c47ffb939f5ca36231d9e3121a252940" # 与 token.csv 里保持一致
```

```
# 生成 kubelet bootstrap kubeconfig 配置文件
kubectl config set-cluster kubernetes \
  --certificate-authority=/opt/kubernetes/ssl/ca.pem \
  --embed-certs=true \
  --server=${KUBE_APISERVER} \
  --kubeconfig=${KUBE_CONFIG}
kubectl config set-credentials "kubelet-bootstrap" \
  --token=${TOKEN} \
  --kubeconfig=${KUBE_CONFIG}
kubectl config set-context default \
  --cluster=kubernetes \
  --user="kubelet-bootstrap" \
  --kubeconfig=${KUBE_CONFIG}
kubectl config use-context default --kubeconfig=${KUBE_CONFIG}
```

4. systemd 管理 kubelet

```
cat > /usr/lib/systemd/system/kubelet.service << EOF
[Unit]
Description=Kubernetes Kubelet
After=docker.service

[Service]
EnvironmentFile=/opt/kubernetes/cfg/kubelet.conf
ExecStart=/opt/kubernetes/bin/kubelet ${KUBELET_OPTS}
Restart=on-failure
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
EOF
```

5. 启动并设置开机启动

```
systemctl daemon-reload
systemctl start kubelet
systemctl enable kubelet
```

5.3 批准 kubelet 证书申请并加入集群

```
# 查看 kubelet 证书请求
kubectl get csr
NAME                                     AGE      SIGNERNAME
REQUESTOR                               CONDITION
node-csr-uCEGP0IiDd1LODKts8J658HrFq9CZ--K6M4G7bjhk8A  6m3s    kubernetes.io/kube-
apiserver-client-kubelet  kubelet-bootstrap  Pending

# 批准申请
kubectl certificate approve node-csr-uCEGP0IiDd1LODKts8J658HrFq9CZ--K6M4G7bjhk8A

# 查看节点
kubectl get node
NAME          STATUS    ROLES    AGE   VERSION
k8s-master1  NotReady  <none>   7s    v1.18.3
```

注：由于网络插件还没有部署，节点会没有准备就绪 NotReady

5.4 部署 kube-proxy

1. 创建配置文件

```
cat > /opt/kubernetes/cfg/kube-proxy.conf << EOF
KUBE_PROXY_OPTS="--logtostderr=false \\\
--v=2 \\\
--log-dir=/opt/kubernetes/logs \\\
--config=/opt/kubernetes/cfg/kube-proxy-config.yml"
EOF
```

2. 配置参数文件

```
cat > /opt/kubernetes/cfg/kube-proxy-config.yml << EOF
kind: KubeProxyConfiguration
apiVersion: kubeproxy.config.k8s.io/v1alpha1
bindAddress: 0.0.0.0
metricsBindAddress: 0.0.0.0:10249
clientConnection:
  kubeconfig: /opt/kubernetes/cfg/kube-proxy.kubeconfig
hostnameOverride: k8s-master1
clusterCIDR: 10.244.0.0/16
EOF
```

3. 生成 kube-proxy.kubeconfig 文件

```
# 切换工作目录
cd ~/TLS/k8s

# 创建证书请求文件
cat > kube-proxy-csr.json << EOF
{
  "CN": "system:kube-proxy",
  "hosts": [],
  "key": {
    "algo": "rsa",
    "size": 2048
  },
  "names": [
    {
      "C": "CN",
      "L": "BeiJing",
      "ST": "BeiJing",
      "O": "k8s",
      "OU": "System"
    }
  ]
}
EOF

# 生成证书
cfssl gencert -ca=ca.pem -ca-key=ca-key.pem -config=ca-config.json -
profile=kubernetes kube-proxy-csr.json | cfssljson -bare kube-proxy
生成 kubeconfig 文件:
KUBE_CONFIG="/opt/kubernetes/cfg/kube-proxy.kubeconfig"
KUBE_APISERVER="https://192.168.31.71:6443"

kubectl config set-cluster kubernetes \
  --certificate-authority=/opt/kubernetes/ssl/ca.pem \
  --embed-certs=true \
  --server=${KUBE_APISERVER} \
  --kubeconfig=${KUBE_CONFIG}
kubectl config set-credentials kube-proxy \
  --client-certificate=./kube-proxy.pem \
  --client-key=./kube-proxy-key.pem \
```

```
--embed-certs=true \  
--kubeconfig=${KUBE_CONFIG}  
kubectl config set-context default \  
--cluster=kubernetes \  
--user=kube-proxy \  
--kubeconfig=${KUBE_CONFIG}  
kubectl config use-context default --kubeconfig=${KUBE_CONFIG}
```

4. systemd 管理 kube-proxy

```
cat > /usr/lib/systemd/system/kube-proxy.service << EOF  
[Unit]  
Description=Kubernetes Proxy  
After=network.target  
  
[Service]  
EnvironmentFile=/opt/kubernetes/cfg/kube-proxy.conf  
ExecStart=/opt/kubernetes/bin/kube-proxy \${KUBE_PROXY_OPTS}  
Restart=on-failure  
LimitNOFILE=65536  
  
[Install]  
WantedBy=multi-user.target  
EOF
```

5. 启动并设置开机启动

```
systemctl daemon-reload  
systemctl start kube-proxy  
systemctl enable kube-proxy
```

5.5 部署网络组件

Calico 是一个纯三层的数据中心网络方案，是目前 Kubernetes 主流的网络方案。

部署 Calico:

```
kubectl apply -f calico.yaml  
kubectl get pods -n kube-system
```

等 Calico Pod 都 Running，节点也会准备就绪:


```
kubectl get node
NAME          STATUS    ROLES    AGE   VERSION
k8s-master    Ready    <none>   37m   v1.20.4
```

5.6 授权 apiserver 访问 kubelet

应用场景: 例如 `kubectl logs`

```
cat > apiserver-to-kubelet-rbac.yaml << EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "true"
  labels:
    kubernetes.io/bootstrapping: rbac-defaults
  name: system:kube-apiserver-to-kubelet
rules:
- apiGroups:
  - ""
  resources:
  - nodes/proxy
  - nodes/stats
  - nodes/log
  - nodes/spec
  - nodes/metrics
  - pods/log
  verbs:
  - "*"
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: system:kube-apiserver
  namespace: ""
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: system:kube-apiserver-to-kubelet
subjects:
```

```
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: kubernetes
EOF

kubectl apply -f apiserver-to-kubelet-rbac.yaml
```

5.7 新增加 Worker Node

1. 拷贝已部署好的 Node 相关文件到新节点

在 Master 节点将 Worker Node 涉及文件拷贝到新节点 192.168.31.72/73

```
scp -r /opt/kubernetes root@192.168.31.72:/opt/

scp -r /usr/lib/systemd/system/{kubelet,kube-proxy}.service
root@192.168.31.72:/usr/lib/systemd/system

scp /opt/kubernetes/ssl/ca.pem root@192.168.31.72:/opt/kubernetes/ssl
```

2. 删除 kubelet 证书和 kubeconfig 文件

```
rm -f /opt/kubernetes/cfg/kubelet.kubeconfig
rm -f /opt/kubernetes/ssl/kubelet*
```

注：这几个文件是证书申请审批后自动生成的，每个 Node 不同，必须删除

3. 修改主机名

```
vi /opt/kubernetes/cfg/kubelet.conf
--hostname-override=k8s-node1

vi /opt/kubernetes/cfg/kube-proxy-config.yml
hostnameOverride: k8s-node1
```

4. 启动并设置开机启动

```
systemctl daemon-reload
systemctl start kubelet kube-proxy
systemctl enable kubelet kube-proxy
```

5. 在 Master 上批准新 Node kubelet 证书申请

```
# 查看证书请求
kubectl get csr
NAME          AGE    SIGNERNAME              REQUESTOR             CONDITION
node-csr-4zTjsaVSRhuyhIGqsefxzVoZDCNKei-aE2jyTP81Uro    89s    kubernetes.io/kube-
apiserver-client-kubelet    kubelet-bootstrap    Pending

# 授权请求
kubectl certificate approve node-csr-4zTjsaVSRhuyhIGqsefxzVoZDCNKei-aE2jyTP81Uro
```

6. 查看 Node 状态

```
kubectl get node
NAME          STATUS    ROLES    AGE    VERSION
k8s-master1   Ready     <none>    47m    v1.20.4
k8s-node1     Ready     <none>    6m49s   v1.20.4
```

Node2 (192.168.31.73) 节点同上。记得修改主机名!

六、部署 Dashboard 和 CoreDNS

6.1 部署 Dashboard

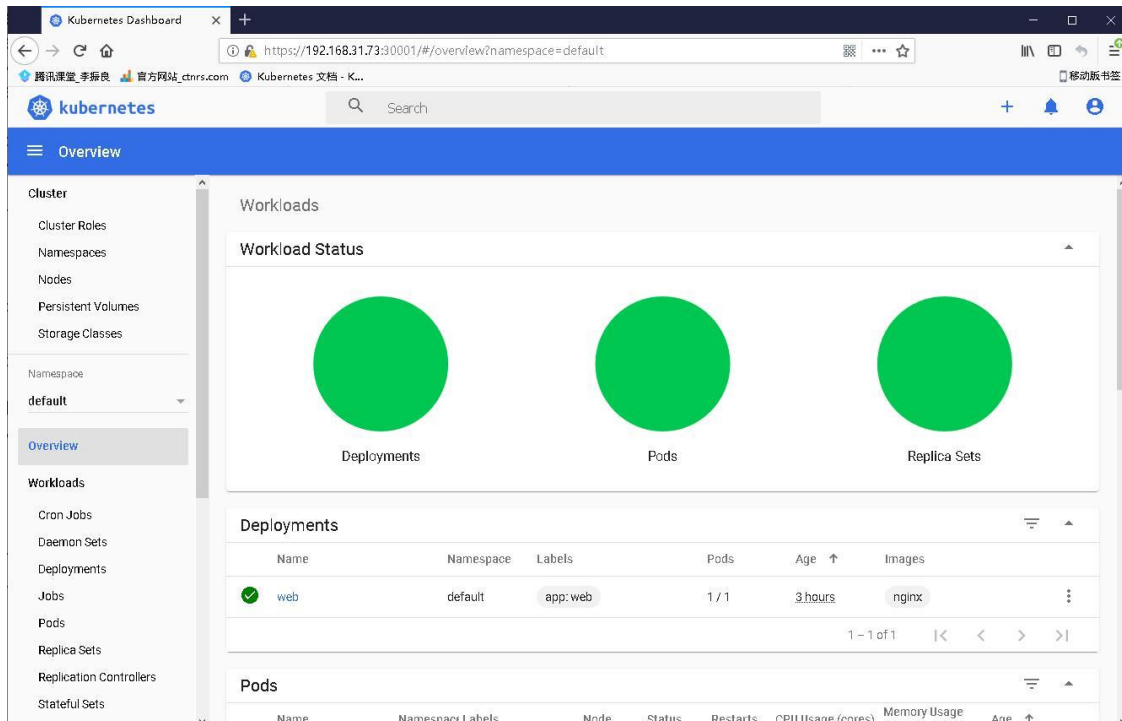
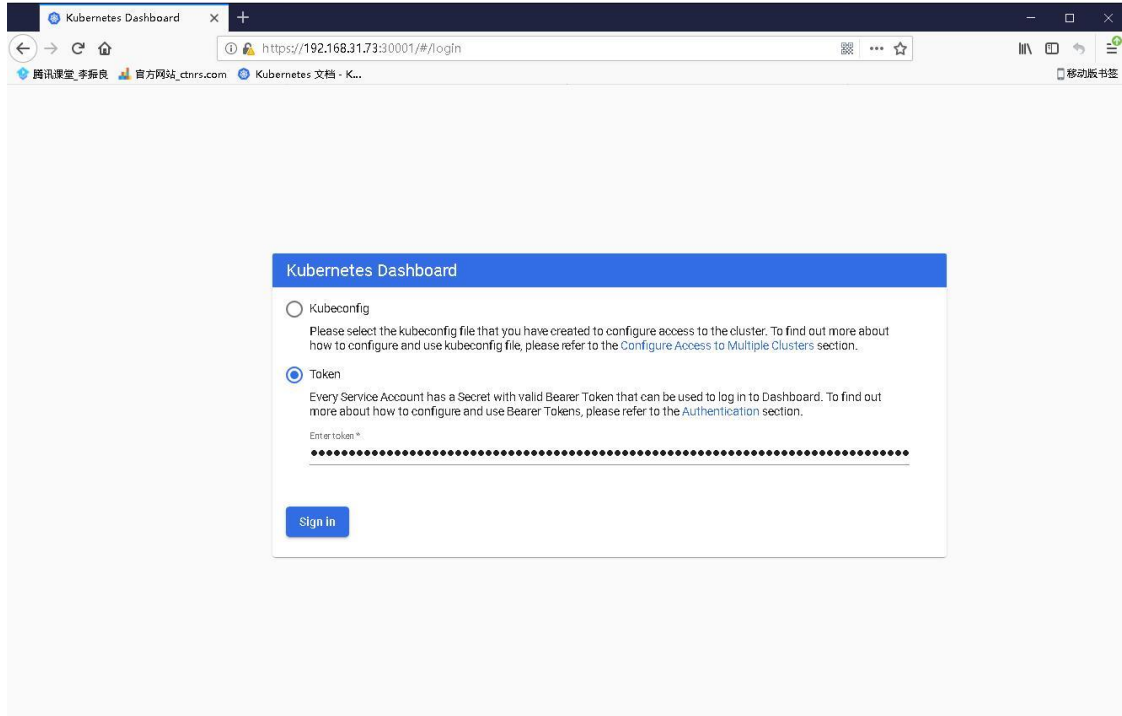
```
kubectl apply -f kubernetes-dashboard.yaml
# 查看部署
kubectl get pods,svc -n kubernetes-dashboard
```

访问地址: <https://NodeIP:30001>

创建 service account 并绑定默认 cluster-admin 管理员集群角色:

```
kubectl create serviceaccount dashboard-admin -n kube-system
kubectl create clusterrolebinding dashboard-admin --clusterrole=cluster-admin --
serviceaccount=kube-system:dashboard-admin
kubectl describe secrets -n kube-system $(kubectl -n kube-system get secret | awk
'/dashboard-admin/{print $1}')
```

使用输出的 token 登录 Dashboard。



6.2 部署 CoreDNS

CoreDNS 用于集群内部 Service 名称解析。

```
kubectl apply -f coredns.yaml
```

```
kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
coredns-5ffbfd976d-j6shb	1/1	Running	0	32s

DNS 解析测试:

```
kubectl run -it --rm dns-test --image=busybox:1.28.4 sh
```

If you don't see a command prompt, try pressing enter.

```
/ # nslookup kubernetes
```

```
Server:      10.0.0.2
```

```
Address 1: 10.0.0.2 kube-dns.kube-system.svc.cluster.local
```

```
Name:        kubernetes
```

```
Address 1: 10.0.0.1 kubernetes.default.svc.cluster.local
```

解析没问题。

至此一个单 Master 集群就搭建完成了！这个环境就足以满足学习实验了，如果你的服务器配置较高，可继续扩容多 Master 集群！

七、扩容多 Master（高可用架构）

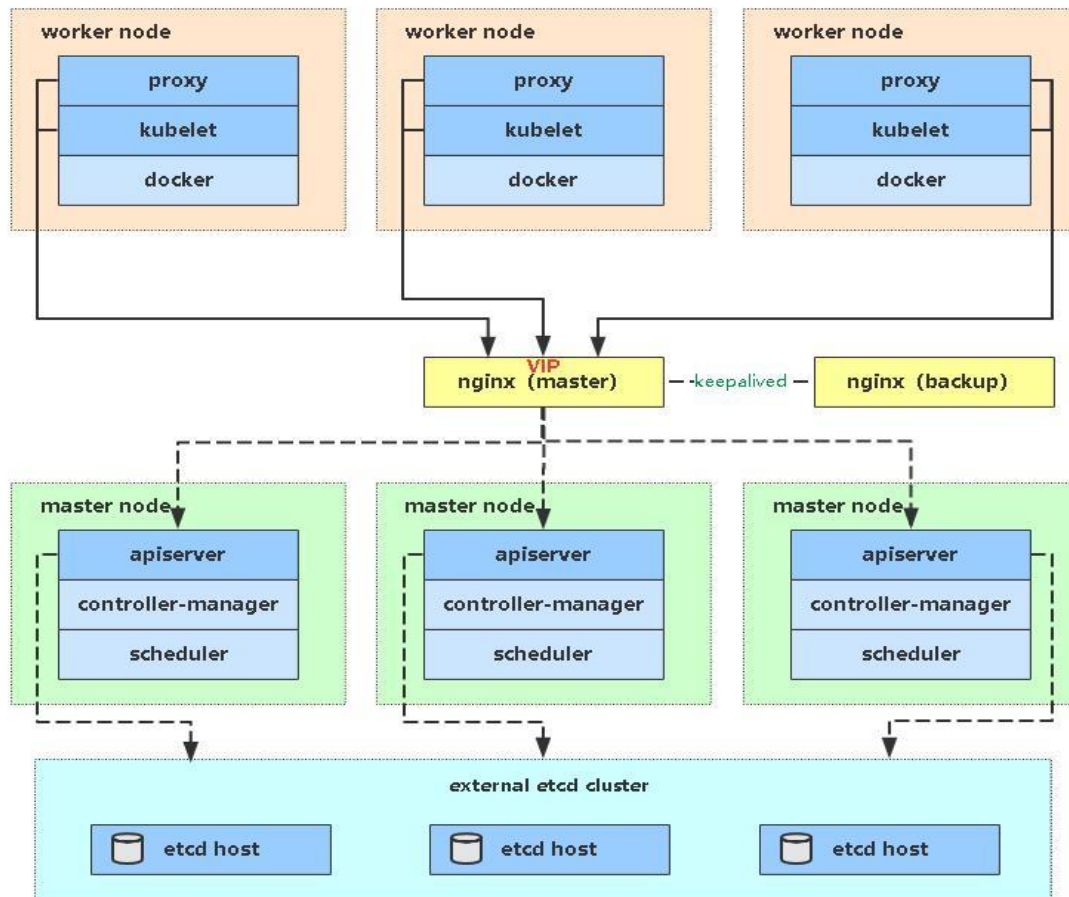
Kubernetes 作为容器集群系统，通过健康检查+重启策略实现了 Pod 故障自我修复能力，通过调度算法实现将 Pod 分布式部署，并保持预期副本数，根据 Node 失效状态自动在其他 Node 拉起 Pod，实现了应用层的高可用性。

针对 Kubernetes 集群，高可用性还应包含以下两个层面的考虑：Etcd 数据库的高可用性和 Kubernetes Master 组件的高可用性。而 Etcd 我们已经采用 3 个节点组建集群实现高可用，本节将对 Master 节点高可用进行说明和实施。

Master 节点扮演着总控中心的角色，通过不断与工作节点上的 Kubelet 和 kube-proxy 进行通信来维护整个集群的健康工作状态。如果 Master 节点故障，将无法使用 kubectl 工具或者 API 做任何集群管理。

Master 节点主要有三个服务 kube-apiserver、kube-controller-manager 和 kube-scheduler，其中 kube-controller-manager 和 kube-scheduler 组件自身通过选择机制已经实现了高可用，所以 Master 高可用主要针对 kube-apiserver 组件，而该组件是以 HTTP API 提供服务，因此对他高可用与 Web 服务器类似，增加负载均衡器对其负载均衡即可，并且可水平扩容。

多 Master 架构图：



7.1 部署 Master2 Node

现在需要再增加一台新服务器，作为 Master2 Node，IP 是 192.168.31.74。

为了节省资源你也可以将之前部署好的 Worker Node1 复用为 Master2 Node 角色（即部署 Master 组件）

Master2 与已部署的 Master1 所有操作一致。所以我们只需将 Master1 所有 K8s 文件拷贝过来, 再修改下服务器 IP 和主机名启动即可。

1. 安装 Docker

```
scp /usr/bin/docker* root@192.168.31.74:/usr/bin
scp /usr/bin/runc root@192.168.31.74:/usr/bin
scp /usr/bin/containerd* root@192.168.31.74:/usr/bin
scp /usr/lib/systemd/system/docker.service
root@192.168.31.74:/usr/lib/systemd/system
scp -r /etc/docker root@192.168.31.74:/etc

# 在 Master2 启动 Docker
systemctl daemon-reload
systemctl start docker
systemctl enable docker
```

2. 创建 etcd 证书目录

在 Master2 创建 etcd 证书目录:

```
mkdir -p /opt/etcd/ssl
```

3. 拷贝文件 (Master1 操作)

拷贝 Master1 上所有 K8s 文件和 etcd 证书到 Master2:

```
scp -r /opt/kubernetes root@192.168.31.74:/opt
scp -r /opt/etcd/ssl root@192.168.31.74:/opt/etcd
scp /usr/lib/systemd/system/kube* root@192.168.31.74:/usr/lib/systemd/system
scp /usr/bin/kubect1 root@192.168.31.74:/usr/bin
scp -r ~/.kube root@192.168.31.74:~
```

4. 删除证书文件

删除 kubelet 证书和 kubeconfig 文件:

```
rm -f /opt/kubernetes/cfg/kubelet.kubeconfig
rm -f /opt/kubernetes/ssl/kubelet*
```

5. 修改配置文件 IP 和主机名

修改 apiserver、kubelet 和 kube-proxy 配置文件为本地 IP:

```
vi /opt/kubernetes/cfg/kube-apiserver.conf
...
--bind-address=192.168.31.74 \
--advertise-address=192.168.31.74 \
...

vi /opt/kubernetes/cfg/kube-controller-manager.kubeconfig
server: https://192.168.31.74:6443

vi /opt/kubernetes/cfg/kube-scheduler.kubeconfig
server: https://192.168.31.74:6443

vi /opt/kubernetes/cfg/kubelet.conf
--hostname-override=k8s-master2

vi /opt/kubernetes/cfg/kube-proxy-config.yml
hostnameOverride: k8s-master2

vi ~/.kube/config
...
server: https://192.168.31.74:6443
```

6. 启动设置开机启动

```
systemctl daemon-reload
systemctl start kube-apiserver kube-controller-manager kube-scheduler kubelet kube-
proxy
systemctl enable kube-apiserver kube-controller-manager kube-scheduler kubelet kube-
proxy
```

7. 查看集群状态

```
kubect1 get cs
```

NAME	STATUS	MESSAGE	ERROR
scheduler	Healthy	ok	
controller-manager	Healthy	ok	
etcd-1	Healthy	{"health": "true"}	

etcd-2	Healthy	{"health": "true"}
etcd-0	Healthy	{"health": "true"}

8. 批准 kubelet 证书申请

```
# 查看证书请求
kubectl get csr
NAME                                AGE          SIGNERNAME          REQUESTOR
CONDITION
node-csr-JYNknakEa_YpHz797oKaN-ZTk43nD51Zc9CJkBLcASU  85m    kubernetes.io/kube-
apiserver-client-kubelet  kubelet-bootstrap  Pending

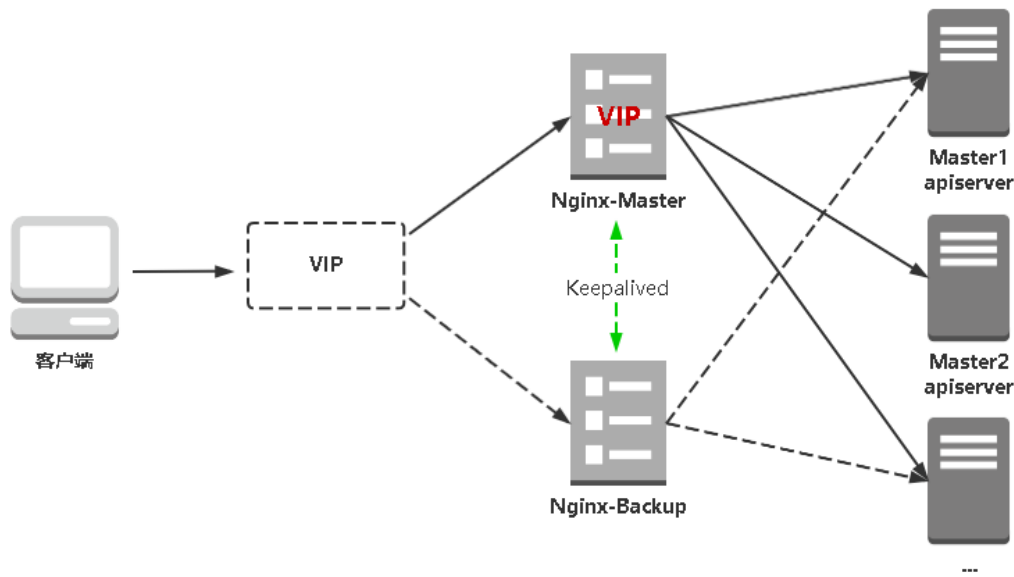
# 授权请求
kubectl certificate approve node-csr-JYNknakEa_YpHz797oKaN-ZTk43nD51Zc9CJkBLcASU

# 查看 Node
kubectl get node
NAME           STATUS    ROLES    AGE   VERSION
k8s-master1   Ready     <none>   34h   v1.20.4
k8s-master2   Ready     <none>   2m    v1.20.4
k8s-node1     Ready     <none>   33h   v1.20.4
k8s-node2     Ready     <none>   33h   v1.20.4
```

如果你在学习遇到问题或者文档有误可联系阿良~ 微信: k8init

7.2 部署 Nginx+Keepalived 高可用负载均衡器

kube-apiserver 高可用架构图:



- Nginx 是一个主流 Web 服务和反向代理服务器，这里用四层实现对 apiserver 实现负载均衡。
- Keepalived 是一个主流高可用软件，**基于 VIP 绑定实现服务器双机热备**，在上述拓扑中，Keepalived 主要根据 Nginx 运行状态判断是否需要故障转移（漂移 VIP），例如当 Nginx 主节点挂掉，VIP 会自动绑定在 Nginx 备节点，从而保证 VIP 一直可用，实现 Nginx 高可用。

注 1：为了节省机器，这里与 K8s Master 节点机器复用。也可以独立于 k8s 集群之外部署，只要 nginx 与 apiserver 能通信就行。

注 2：如果你是在公有云上，一般都不支持 keepalived，那么你可以直接用它们的负载均衡器产品，直接负载均衡多台 Master kube-apiserver，架构与上面一样。

在两台 Master 节点操作。

1. 安装软件包（主/备）

```
yum install epel-release -y
yum install nginx keepalived -y
```

2. Nginx 配置文件（主/备一样）

```
cat > /etc/nginx/nginx.conf << "EOF"
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;

include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
}

# 四层负载均衡，为两台 Master apiserver 组件提供负载均衡
stream {

    log_format main '$remote_addr $upstream_addr - [$time_local] $status
$upstream_bytes_sent';

    access_log /var/log/nginx/k8s-access.log main;

    upstream k8s-apiserver {
        server 192.168.31.71:6443; # Master1 APISERVER IP:PORT
        server 192.168.31.74:6443; # Master2 APISERVER IP:PORT
    }

    server {
        listen 16443; # 由于 nginx 与 master 节点复用，这个监听端口不能是 6443，否则会
冲突
        proxy_pass k8s-apiserver;
    }
}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
'$status $body_bytes_sent "$http_referer" '
'$http_user_agent' "$http_x_forwarded_for";

    access_log /var/log/nginx/access.log main;
```

```
sendfile            on;
tcp_nopush          on;
tcp_nodelay         on;
keepalive_timeout   65;
types_hash_max_size 2048;

include             /etc/nginx/mime.types;
default_type        application/octet-stream;

server {
    listen          80 default_server;
    server_name     _;

    location / {
    }
}
EOF
```

3. keepalived 配置文件 (Nginx Master)

```
cat > /etc/keepalived/keepalived.conf << EOF
global_defs {
    notification_email {
        acassen@firewall.loc
        failover@firewall.loc
        sysadmin@firewall.loc
    }
    notification_email_from Alexandre.Cassen@firewall.loc
    smtp_server 127.0.0.1
    smtp_connect_timeout 30
    router_id NGINX_MASTER
}

vrrp_script check_nginx {
    script "/etc/keepalived/check_nginx.sh"
}

vrrp_instance VI_1 {
    state MASTER
    interface ens33 # 修改为实际网卡名
```

```
virtual_router_id 51 # VRRP 路由 ID 实例，每个实例是唯一的
priority 100      # 优先级，备服务器设置 90
advert_int 1      # 指定 VRRP 心跳包通告间隔时间，默认 1 秒
authentication {
    auth_type PASS
    auth_pass 1111
}
# 虚拟 IP
virtual_ipaddress {
    192.168.31.88/24
}
track_script {
    check_nginx
}
}
EOF
```

- `vrp_script`: 指定检查 nginx 工作状态脚本（根据 nginx 状态判断是否故障转移）
- `virtual_ipaddress`: 虚拟 IP (VIP)

准备上述配置文件中检查 nginx 运行状态的脚本:

```
cat > /etc/keepalived/check_nginx.sh << "EOF"
#!/bin/bash
count=$(ss -antp |grep 16443 |egrep -cv "grep|$$")

if [ "$count" -eq 0 ];then
    exit 1
else
    exit 0
fi
EOF
chmod +x /etc/keepalived/check_nginx.sh
```

4. keepalived 配置文件 (Nginx Backup)

```
cat > /etc/keepalived/keepalived.conf << EOF
global_defs {
    notification_email {
```

```
    acassen@firewall.loc
    failover@firewall.loc
    sysadmin@firewall.loc
}
notification_email_from Alexandre.Cassen@firewall.loc
smtp_server 127.0.0.1
smtp_connect_timeout 30
router_id NGINX_BACKUP
}

vrrp_script check_nginx {
    script "/etc/keepalived/check_nginx.sh"
}

vrrp_instance VI_1 {
    state BACKUP
    interface ens33
    virtual_router_id 51 # VRRP 路由 ID 实例, 每个实例是唯一的
    priority 90
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 1111
    }
    virtual_ipaddress {
        192.168.31.88/24
    }
    track_script {
        check_nginx
    }
}
EOF
```

准备上述配置文件中检查 nginx 运行状态的脚本:

```
cat > /etc/keepalived/check_nginx.sh << "EOF"
#!/bin/bash
count=$(ss -antp |grep 16443 |egrep -cv "grep|$$")

if [ "$count" -eq 0 ];then
```

```
    exit 1
else
    exit 0
fi
EOF
chmod +x /etc/keepalived/check_nginx.sh
```

注: keepalived 根据脚本返回状态码 (0 为工作正常, 非 0 不正常) 判断是否故障转移。

5. 启动并设置开机启动

```
systemctl daemon-reload
systemctl start nginx keepalived
systemctl enable nginx keepalived
```

6. 查看 keepalived 工作状态

```
ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group
default qlen 1000
    link/ether 00:0c:29:04:f7:2c brd ff:ff:ff:ff:ff:ff
    inet 192.168.31.80/24 brd 192.168.31.255 scope global noprefixroute ens33
        valid_lft forever preferred_lft forever
    inet 192.168.31.88/24 scope global secondary ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe04:f72c/64 scope link
        valid_lft forever preferred_lft forever
```

可以看到, 在 ens33 网卡绑定了 192.168.31.88 虚拟 IP, 说明工作正常。

7. Nginx+Keepalived 高可用测试

关闭主节点 Nginx, 测试 VIP 是否漂移到备节点服务器。

在 Nginx Master 执行 `kill nginx`;
在 Nginx Backup, `ip addr` 命令查看已成功绑定 VIP。

8. 访问负载均衡器测试

找 K8s 集群中任意一个节点, 使用 `curl` 查看 K8s 版本测试, 使用 VIP 访问:

```
curl -k https://192.168.31.88:16443/version
{
  "major": "1",
  "minor": "20",
  "gitVersion": "v1.20.4",
  "gitCommit": "e87da0bd6e03fea7933c4b5263d151aafd07c",
  "gitTreeState": "clean",
  "buildDate": "2021-02-18T16:03:00Z",
  "goVersion": "go1.15.8",
  "compiler": "gc",
  "platform": "linux/amd64"
}
```

可以正确获取到 K8s 版本信息, 说明负载均衡器搭建正常。该请求数据流程: `curl` → `vip(nginx)` → `apiserver`

通过查看 Nginx 日志也可以看到转发 `apiserver` IP:

```
tail /var/log/nginx/k8s-access.log -f
192.168.31.71 192.168.31.71:6443 - [02/Apr/2021:19:17:57 +0800] 200 423
192.168.31.71 192.168.31.72:6443 - [02/Apr/2021:19:18:50 +0800] 200 423
```

到此还没结束, 还有下面最关键的一步。

7.3 修改所有 Worker Node 连接 LB VIP

试想下, 虽然我们增加了 Master2 Node 和负载均衡器, 但是我们是从单 Master 架构扩容的, 也就是说目前所有的 Worker Node 组件连接都还是 Master1 Node, 如果不改为连接 VIP 走负载均衡器, 那么 Master 还是单点故障。

因此接下来就是要改所有 Worker Node (`kubectl get node` 命令查看到的节点) 组件配置文件, 由原来 192.168.31.71 修改为 192.168.31.88 (VIP)。

在所有 Worker Node 执行:


```
sed -i 's#192.168.31.71:6443#192.168.31.88:16443#' /opt/kubernetes/cfg/*  
systemctl restart kubelet kube-proxy
```

检查节点状态:

```
kubectl get node  
NAME           STATUS    ROLES    AGE   VERSION  
k8s-master1    Ready     <none>    32d   v1.20.4  
k8s-master2    Ready     <none>    10m   v1.20.4  
k8s-node1      Ready     <none>    31d   v1.20.4  
k8s-node2      Ready     <none>    31d   v1.20.4
```

至此, 一套完整的 Kubernetes 高可用集群就部署完成了!



阿良个人微信



DevOps技术栈公众号